Recitation 06: Read-aloud Teaching Notes

Recitation 06：可直接朗读的讲课讲义

Today we will practice functions and strings. Read each English explanation together with its code, and use the Chinese immediately below it as a reference.

今天我们练习函数和字符串。请结合代码阅读每段英文讲解，并用其正下方的中文作为参考。

The runnable examples use editable sample values rather than waiting for keyboard input. The original interactive tasks are preserved, and we will point out where input calls belong.

可运行示例使用可编辑样例值，不等待键盘输入。原题的交互要求保留，我们也会指出输入调用应放在哪里。

Opening: four ideas about functions

开场：函数的四个概念

A function gives a name to a small task. Defining it describes what to do; calling it performs that task once. A def statement alone does not run the function body.

函数给一个小任务起名字。定义描述怎么做，调用才执行一次任务。单独执行 def 语句不会运行函数体。

A parameter is a name in the definition, while an argument is a value supplied at the call. In def square(number), number is a parameter; in square(5), 5 is an argument.

形参是定义中的名字，实参是调用时提供的值。在 def square(number) 中，number 是形参；在 square(5) 中，5 是实参。

Printing is for display; returning is for reuse. return sends a value back to the caller and ends that call immediately. A function that does not execute a return statement with a value returns None, even if it printed something.

打印用于显示，返回用于复用。return 把一个值交回调用者，并立即结束该次调用。函数若没有执行带值的 return，即使打印过内容，也会返回 None。

A local variable belongs to the function call, while a global variable is defined outside the function. Assigning to a name inside a function normally makes it local; changing a same-named global requires an explicit global declaration.

局部变量属于函数调用，全局变量则定义在函数外。在函数内给名字赋值通常会使它成为局部变量；修改同名全局变量需要明确的 global 声明。

For each exercise, first identify the inputs and outputs. Then choose parameters and return values, write the conditions or loops, and finally call the function and check an example.

每道题先确定输入和输出，再选择形参与返回值、编写条件或循环，最后调用函数并核对示例。

Source issues and stated assumptions

原题问题与明确假设

In Exercise 1 Program 2, use cal consistently; calc in the source definition is a typo. Program 3 first raises UnboundLocalError. Print each predicted answer directly, then show corrected code in a separate block below.

第 1 题程序 2 统一使用 cal；原题定义中的 calc 是笔误。程序 3 首先出现 UnboundLocalError。每个答案直接打印，再在下方独立代码块给出修正程序。

Exercise 5 refers to a grading table that is absent from the PDF. We explicitly assume scores from 0 to 100 with A/B/C/D boundaries at 90/80/70/60; these are not source-provided thresholds.

第 5 题提到了 PDF 中缺失的评分表。这里明确假设分数为 0 到 100，A/B/C/D 的分界为 90/80/70/60；这些不是原题给出的分界。

Exercise 3 uses fractional gallons, Exercise 8 assumes a valid date, and Exercise 9 uses .?! as sentence boundaries. Exercise 10 counts every character case-sensitively, keeps the first character on a tie, and counts vowels in either case.

第 3 题使用小数加仑，第 8 题假设日期有效，第 9 题使用 .?! 作为句界。第 10 题区分大小写统计所有字符，平局保留最早字符，元音则不区分大小写。

Exercise 11 uses an inclusive non-negative range and shows N/A for an undefined mean. In Exercise 12, the source example ending in X conflicts with the stated formula: its total is 46 and its required check character is 2.

第 11 题使用包含两端的非负范围，平均数无定义时显示 N/A。第 12 题末位为 X 的原样例与给定公式冲突：加权总和为 46，所需校验字符为 2。

Exercise 1 | Predict the output

练习 1｜预测输出

Predict the output of each original program. If execution stops with an error, identify the first error reached.

预测每个原程序的输出。如果执行因错误停止，请指出首先到达的错误。

Program 1

程序 1

```
def f():
    global s
    print(s)
    s = "more"
    return s
    s = "less"
s = "less"
print(f())
print(s)
```

Program 2

程序 2

```
def cal(op, x, y):
    return op(x, y)
def add(a, b):
    return a + b
def div(a, b):
    if b != 0:
        return a/b
    print('Denominator was 0.')

print(cal(add, 2, 3))
print(cal(div, 2, 0))
```

Program 3

程序 3

```
def foo(x, y):
    a += 1
    x = a
    y = a**2
    return x, y
a = 2
x, y = foo('a', 'b')
print(a, b, x, y)
```

Teaching walkthrough

逐步讲解

Let us trace each program in execution order. For Program 2, use cal as the function name throughout.

我们按执行顺序跟踪每个程序。程序 2 的函数名统一使用 cal。

In Program 1, defining f does not run its body. The assignment before the call sets the global s to "less".

在程序 1 中，定义 f 不会运行函数体。调用前的赋值把全局 s 设为 "less"。

Inside f, global s refers to that global variable. The inner print displays less, and the assignment changes s to more.

在 f 内，global s 指向这个全局变量。内部打印先显示 less，随后赋值把 s 改为 more。

return sends more to the caller and ends the call. The outer print displays more, and the final print displays the updated global more; the assignment after return never runs.

return 把 more 交回调用者并结束调用。外层打印显示 more，最后的打印显示更新后的全局 more；return 后的赋值不会执行。

In Program 2, cal receives a function as op and calls op(x, y). Passing add produces 5.

在程序 2 中，cal 通过 op 接收一个函数，再调用 op(x, y)。传入 add 会得到 5。

Passing div with a zero denominator prints Denominator was 0. No return statement with a value runs on that path, so the outer print then displays None.

传入 div 且分母为零时，会打印 Denominator was 0.。这条执行路径没有运行带值的 return，因此外层打印接着显示 None。

In Program 3, a += 1 both reads and assigns a. Assignment makes a local to foo, but that local variable has not been assigned a value, so the first error is UnboundLocalError.

在程序 3 中，a += 1 既读取又赋值 a。赋值使 a 成为 foo 的局部变量，但这个局部变量还没有值，因此首先出现 UnboundLocalError。

The final print is not reached. It also refers to an undefined b: the string argument 'b' does not create a variable named b.

最后的打印不会到达。它还引用了未定义的 b：字符串实参 'b' 不会创建名为 b 的变量。

The answer cell prints the predicted error message directly. The next code cell contains the corrected program, which takes the initial value as a parameter and prints only defined names.

答案单元格直接打印预测的报错信息。下一个代码单元格给出修正程序：把初始值作为参数传入，并且只打印已经定义的名字。

Program 1 answer

程序 1 的答案

The output is `less`, `more`, `more`, one per line. The inner print runs before the outer print. `global s` changes the global value, and the assignment after `return` is unreachable.

输出依次为 less、more、more，每项单独一行。内部打印先于外部打印运行。global s 会改变全局值，return 后的赋值语句无法到达。

```
# Program 1: the original program.
# 程序 1：原题程序。
# Define a function with no parameters; its body runs only when called.
# 定义无参数函数；函数体直到调用时才执行。
def f():
    # Use the global variable s inside this function.
    # 指明函数中的 s 使用全局变量。
    global s
    # The global value is less at this point, so print less first.
    # 此时全局 s 为 less，因此先显示 less。
    print(s)
    # Change the global value to more; the change is also visible outside.
    # 把全局变量改为 more，函数外也能看到变化。
    s = "more"
```

```
    # Return more and end this function call immediately.
    # 返回 more 并立即结束这次函数调用。
    return s
    # This statement follows return and can never run.
    # 这一行在 return 后面，永远无法执行。
    s = "less"
# Create the global string variable before calling the function.
# 调用之前先创建全局字符串变量。
s = "less"
# Run the inner print first, then print the returned value more.
# 先执行函数内部打印，再打印函数返回的 more。
print(f())
# Display the updated global value more.
# 显示修改后的全局值 more。
print(s)
```

```
less
more
more
```

Program 2 answer

程序 2 的答案

The output is 5, `Denominator was 0.`, and `None`, one per line. The answer cell prints these three lines directly.

输出依次为 5、`Denominator was 0.` 和 `None`，每项单独一行。答案单元格直接打印这三行。

```
# Print the result of adding 2 and 3.
# 打印二加三得到的结果。
print(5)
# Print the message produced by div when its denominator is zero.
# 打印 div 遇到零分母时显示的提示。
print("Denominator was 0.")
# Print the implicit return value of that zero-denominator call.
# 打印这次零分母调用的隐式返回值。
print(None)
```

```
5
Denominator was 0.
None
```

Program 2: corrected code

程序 2：修正代码

Use `cal` consistently. Running this full program produces the three answer lines above.

统一使用 `cal`。运行下面的完整程序，会产生上方答案中的三行输出。

```
# Receive a function and two values; use the corrected name cal.
# 接收一个函数和两个值；函数名按笔误修正为 cal。
def cal(op, x, y):
    # Call the supplied function and return its result.
    # 调用传入的函数并返回其结果。
    return op(x, y)

# Define the addition operation.
# 定义加法运算。
def add(a, b):
    # Return the sum of the two arguments.
    # 返回两个实参的和。
    return a + b
```

```
# Define the division operation.
# 定义除法运算。
def div(a, b):
    # Divide only when the denominator is not zero.
    # 仅在分母非零时执行除法。
    if b != 0:
        # Return the quotient on the nonzero path.
        # 在分母非零的路径返回商。
        return a / b
    # Print the message; this path then implicitly returns None.
    # 打印提示；这条路径随后隐式返回 None。
    print("Denominator was 0.")

# Run add through cal and print 5.
# 通过 cal 调用 add，并打印 5。
print(cal(add, 2, 3))
# Run div through cal; its message appears before the outer print displays None.
# 通过 cal 调用 div；内部提示先显示，随后外层打印 None。
print(cal(div, 2, 0))
```

```
5
Denominator was 0.
None
```

Program 3 answer

程序 3 的答案

The first error is `UnboundLocalError` at `a += 1`: assignment makes `a` local before it has a value. The final print is not reached; it also refers to an undefined `b`. The answer cell prints the error message directly.

第一处错误是 `a += 1` 处的 `UnboundLocalError`：赋值让 `a` 成为局部变量，但它还没有值。最后的打印不会到达；它还引用了未定义的 `b`。答案单元格直接打印报错信息。

```
# Print the first error reached by the original program.
# 直接打印原程序首先遇到的报错信息。
print("UnboundLocalError: local variable 'a' referenced before assignment")
```

```
UnboundLocalError: local variable 'a' referenced before assignment
```

Program 3: corrected code

程序 3：修正代码

Pass the initial value explicitly and print only defined names. The output is `2 3 9`.

显式传入初始值，并且只打印已经定义的名字。输出为 `2 3 9`。

```
# Receive the initial value as a local parameter.
# 用局部形参接收初始值。
def foo_fixed(a):
    # The parameter already has a value, so increase it safely.
    # 形参已经有值，因此可以安全地加一。
    a += 1
    # Save the updated value 3.
    # 保存更新后的值 3。
    x = a
    # Calculate its square, 9.
    # 计算它的平方 9。
    y = a ** 2
    # Return both results to the caller.
    # 把两个结果返回调用处。
    return x, y
```

```
# Keep the caller's initial value 2.
# 保存调用者的初始值 2。
a = 2
# Receive the returned values 3 and 9.
# 接收返回的两个值 3 和 9。
x, y = foo_fixed(a)
# Print only defined names; the caller's a remains 2.
# 只打印已经定义的名字；调用者的 a 仍为 2。
print(a, x, y)
```

```
2 3 9
```

Common mistakes

常见错误

Do not confuse printing with returning, execute a line after return, or treat a string argument as a variable definition.

不要混淆打印和返回，不要执行 return 后的语句，也不要把字符串实参当成变量定义。

Check your understanding

检查理解

Question: Why does the outer print display None after div(2, 0) prints its message?

问题：div(2, 0) 打印提示后，为什么外层打印还会显示 None？

Answer: Printing a message does not return a value. This path reaches the end of the function without a return statement with a value, so it returns None.

答案：打印提示不会返回值。这条路径没有运行带值的 return 就到达函数末尾，因此返回 None。

Now let us use functions in a calculation with explicit units.

现在我们把函数用于一个带有明确单位关系的计算。

Exercise 2 | Property Tax

练习 2｜房产税

A city collects property taxes on the assessment value of property, which is 60 percent of the property's actual value. For example, land valued at $10,000 has an assessment value of $6,000. The property tax is 72 cents for each $100 of assessment value, so the tax on $6,000 is $43.20.

某城市按评估值征收房产税，评估值是实际价值的 60%。例如，实际价值 10000 美元的土地，评估值是 6000 美元。每 100 美元评估值收取 72 美分税，因此 6000 美元的税额为 43.20 美元。

Write a program that asks for the actual value of a piece of property and calls a function named `computePropertyTax` to calculate and display the assessment value and property tax.

编写程序，询问房产的实际价值，并调用名为 `computePropertyTax` 的函数计算和显示评估值及房产税。

Teaching walkthrough

逐步讲解

Our goal is to turn percentages and units into expressions, then pass a value into a function.

本题目标是把百分比和单位关系写成表达式，再把一个值传入函数。

Before writing the function, let us check the units. The assessment value is actual_value * 0.60.

写函数之前，我们先核对单位。评估值等于 actual_value * 0.60。

Seventy-two cents is 0.72 dollars, charged for each 100 dollars of assessment value. The tax is therefore assessment_value / 100 * 0.72.

72 美分是 0.72 美元，对应每 100 美元评估值。因此税额是 assessment_value / 100 * 0.72。

Keep the function name computePropertyTax because the task requires it. In its definition, actual_value is a parameter; in our call, 10000.0 is the argument supplied to that parameter.

保留 computePropertyTax 这个函数名，因为题目指定了它。在定义中，actual_value 是形参；在调用中，10000.0 是传给该形参的实参。

The function calculates both values and displays them with two decimal places. Formatting controls the displayed amount; it does not change the tax rule.

函数计算两项数值，并按两位小数显示。格式控制显示的金额，不会改变税额计算规则。

For the source example, 10000 times 0.60 gives 6000, and 6000 divided by 100 times 0.72 gives 43.20. Try changing the input value while leaving the function body unchanged.

对于原题示例，10000 乘 0.60 得到 6000，6000 除以 100 再乘 0.72 得到 43.20。请修改输入值，同时保持函数体不变。

The runnable example uses a fixed editable value so it does not pause. For an interactive program, replace that assignment with float(input("Enter actual value: ")).

可运行示例使用固定的可编辑数值，因此不会暂停。交互程序可以把该赋值改成 float(input("Enter actual value: "))。

Worked code and results

对照代码与运行结果

```
# Accept the actual value and use the required function name to calculate and display.
# 接收实际价值，用题目指定的函数名完成计算和显示。
def computePropertyTax(actual_value):
    # The assessment value is 60 percent of the actual value.
    # 实际价值的百分之六十是评估值。
    assessment_value = actual_value * 0.60
    # Charge 0.72 dollars for every 100 dollars of assessment value.
    # 每一百美元收零点七二美元税。
    property_tax = assessment_value / 100 * 0.72
    # Display the assessment value with two decimal places.
    # 按两位小数显示评估值。
    print(f"Assessment value: ${assessment_value:.2f}")
    # Display the property tax with two decimal places.
    # 按两位小数显示税额。
    print(f"Property tax: ${property_tax:.2f}")

# Use an editable example instead of blocking input; replace it with float(input(...)) for interaction.
# 用可编辑示例代替阻塞式输入；课堂可改为 float(input(...))。
actual_value = 10000.0
# Pass the actual value to the function and display both results.
# 把实际价值传入函数并显示两项结果。
computePropertyTax(actual_value)
```

```
Assessment value: $6000.00
Property tax: $43.20
```

Worked answer

完整解答

Convert 72 cents to $0.72 and apply it per $100 of assessment value. For actual value $10,000, the assessment value is $6,000 and tax is $43.20.

把 72 美分换算成 0.72 美元，并按每 100 美元评估值计算。实际价值为 10000 美元时，评估值是 6000 美元，税额是 43.20 美元。

Common mistakes

常见错误

Do not confuse cents with dollars or forget the division by 100 in the tax rate.

不要混淆美分与美元，也不要漏掉税率中除以 100 的步骤。

Check your understanding

检查理解

Question: What happens to assessment value and tax if the actual value doubles?

问题：实际价值翻倍时，评估值和税额会怎样？

Answer: Both double, because both calculations use fixed proportions.

答案：两者都翻倍，因为两项计算都使用固定比例。

One function just calculated two results. Next, we will split a larger task into five functions and follow the flow of returned values.

刚才一个函数计算了两项结果。接下来把更大任务拆成五个函数，并跟踪返回值的流动。

Exercise 3 | Painting cost

练习 3｜刷漆费用

A painting company needs one gallon of paint and eight hours of labor for every 112 square feet of wall space. Labor costs $35.00 per hour.

某刷漆公司每处理 112 平方英尺墙面，需要 1 加仑油漆和 8 小时劳动。每小时劳动费用为 35.00 美元。

Write a program that asks for the wall area in square feet and the paint price per gallon. Call these functions to calculate and display the results:

编写程序，询问以平方英尺计的墙面面积及每加仑油漆价格。调用以下函数计算并显示结果：

1. `paint_volume`: gallons of paint required.

`paint_volume`：所需油漆加仑数。

2. `labor_hours`: labor hours required.

`labor_hours`：所需劳动小时数。

3. `paint_cost`: cost of the paint.

`paint_cost`：油漆费用。

4. `labor_cost`: labor charges.

`labor_cost`：劳动费用。

5. `total_cost`: total cost of the paint job.

`total_cost`：整个刷漆工作的总费用。

Assumption: use proportional fractional gallons. The source does not ask us to round up purchased gallons.

假设：按比例使用小数加仑。原题没有要求把购买加仑数向上取整。

Teaching walkthrough

逐步讲解

Our goal is to split one task into five small functions and connect their returned values.

本题目标是把一个任务拆成五个小函数，并连接它们的返回值。

Let each function do one small job. Area determines the paint quantity and labor time; those results determine the two charges, and the charges determine the total.

让每个函数只做一件小事。面积决定油漆用量和工时；这两项结果决定两类费用，两类费用再决定总价。

paint_volume returns area / 112. labor_hours returns area / 112 * 8, because every 112 square feet needs eight hours.

paint_volume 返回 area / 112。labor_hours 返回 area / 112 * 8，因为每 112 平方英尺需要 8 小时。

paint_cost accepts the gallons already calculated and the price per gallon. labor_cost accepts the hours, and total_cost accepts the two charges.

paint_cost 接收已经算出的加仑数与每加仑价格。labor_cost 接收工时，total_cost 接收两项费用。

These functions return their calculations rather than only printing them. The caller can then save each result, reuse it in the next step, and display all five answers.

这些函数返回计算结果，而不只是打印。调用者就能保存每项结果、在下一步复用，并显示五项答案。

With 224 square feet and a price of 25 dollars per gallon, we need 2 gallons and 16 hours. The paint costs 50 dollars, labor costs 560 dollars, and the total is 610 dollars.

面积为 224 平方英尺、每加仑 25 美元时，需要 2 加仑和 16 小时。油漆费 50 美元、劳动费 560 美元，总费用 610 美元。

We allow fractional gallons because the source gives no whole-container purchasing rule. For 56 square feet, the same proportions give 0.5 gallons and 4 hours.

原题没有整桶采购规则，所以这里允许小数加仑。对于 56 平方英尺，相同比例得到 0.5 加仑和 4 小时。

Worked code and results

对照代码与运行结果

```
# First, return the required gallons from the wall area.
# 第一步根据墙面面积返回所需加仑数。
def paint_volume(area):
    # Each 112 square feet needs one gallon, so divide proportionally.
    # 一百一十二平方英尺对应一加仑，按比例相除。
    return area / 112
```

```
# Second, return the labor hours for the same area.
# 第二步根据同一面积返回劳动小时数。
def labor_hours(area):
    # Find the number of 112-square-foot units, then multiply by eight hours.
    # 先求面积倍数，再乘每份所需的八小时。
    return area / 112 * 8

# Third, accept the quantity and unit price to calculate paint cost.
# 第三步接收用量和单价，计算油漆费用。
def paint_cost(gallons, price_per_gallon):
    # Multiply the gallons by the price per gallon.
    # 用量乘每加仑价格得到总油漆费。
    return gallons * price_per_gallon

# Fourth, accept the hours and use the stated hourly rate.
# 第四步接收工时，使用题目指定的时薪。
def labor_cost(hours):
    # Multiply the hours by 35 dollars per hour to obtain labor charges.
    # 每小时三十五美元，乘工时得到劳动费。
    return hours * 35.00

# Fifth, combine the two kinds of charges.
# 第五步把两类费用组合起来。
def total_cost(paint_charge, labor_charge):
    # The total is the paint charge plus the labor charge.
    # 总费用等于油漆费与劳动费之和。
    return paint_charge + labor_charge

# Use an editable wall area; the interactive program could use float(input(...)).
# 可编辑墙面面积，原题交互版可从 float(input(...)) 获得。
wall_area = 224.0
# Use an editable price per gallon.
# 可编辑每加仑油漆价格。
price_per_gallon = 25.0
# Call the first function and save the paint quantity.
# 调用第一个函数并保存油漆用量。
gallons = paint_volume(wall_area)
# Call the second function and save the hours.
# 调用第二个函数并保存工时。
hours = labor_hours(wall_area)
# Pass the quantity and unit price to the paint-cost function.
# 把用量与单价传给费用函数。
paint_charge = paint_cost(gallons, price_per_gallon)
# Pass the hours to the labor-cost function.
# 把工时传给劳动费用函数。
labor_charge = labor_cost(hours)
# Pass both charges to the total-cost function.
# 把两项费用传给总价函数。
job_charge = total_cost(paint_charge, labor_charge)
# Display the paint quantity; the example requires two gallons.
# 显示油漆用量，示例为两加仑。
print(f"Paint required: {gallons:.2f} gallons")
# Display the labor time; the example requires sixteen hours.
# 显示工时，示例为十六小时。
print(f"Labor required: {hours:.2f} hours")
# Display the paint cost of fifty dollars.
# 显示油漆费用五十美元。
print(f"Paint cost: ${paint_charge:.2f}")
# Display the labor charge of five hundred sixty dollars.
# 显示劳动费用五百六十美元。
print(f"Labor cost: ${labor_charge:.2f}")
# Display the total charge of six hundred ten dollars.
# 显示总价六百一十美元。
print(f"Total cost: ${job_charge:.2f}")
```

```
Paint required: 2.00 gallons
Labor required: 16.00 hours
Paint cost: $50.00
Labor cost: $560.00
Total cost: $610.00
```

Worked answer

完整解答

Use fractional gallons as stated above. For 224 square feet and $25 per gallon: 2 gallons, 16 hours, $50 paint, $560 labor, and $610 total.

按上面的约定使用小数加仑。面积为 224 平方英尺、每加仑 25 美元时，需要 2 加仑油漆和 16 小时劳动；油漆费 50 美元、劳动费 560 美元，总费用 610 美元。

Common mistakes

常见错误

Do not only print inside a function whose result is needed later, confuse area with gallons, or add an unstated whole-gallon rounding rule.

后续需要结果的函数不能只是打印；不要混淆面积与加仑数，也不要添加原题未要求的整加仑取整规则。

Check your understanding

检查理解

Question: Why does total_cost need only the two charges?

问题：为什么 total_cost 只需要两项费用？

Answer: Earlier functions have already converted paint quantity and labor hours into charges, so the final function only adds them.

答案：前面的函数已经把油漆用量和工时转换成费用，因此最后的函数只需把它们相加。

A function can make a choice as well as calculate a quantity. Let us practice that with two values.

函数既能计算数量，也能作出选择。我们用两个值练习这种选择。

Exercise 4 | Maximum of Two Values

练习 4｜两个值中的较大值

Write `max_of_two_numbers`, which accepts two integer arguments and returns the greater value. For example, passing 7 and 12 should return 12. Use the function in a program that prompts for two integers and displays the greater value.

编写 `max_of_two_numbers`，接收两个整数实参并返回较大值。例如，传入 7 和 12 应返回 12。在程序中使用这个函数，提示输入两个整数并显示较大值。

If the values are equal, return that shared value.

如果两个值相等，返回它们共同的值。

Teaching walkthrough

逐步讲解

Our goal is to put a conditional choice inside a function that returns a value.

本题目标是把条件选择写入一个返回值的函数。

Compare the two values, choose one, and return it. If first >= second, return first; otherwise, return second.

比较两个值，选择其中一个并返回。如果 first >= second，就返回 first；否则返回 second。

Every branch returns a result. The greater-than-or-equal comparison also makes the equal-input case explicit.

每个分支都返回结果。大于或等于比较也明确处理了两个输入相等的情况。

For 7 and 12, the condition is false and the function returns 12. For 12 and 7, the first branch returns 12; for 7 and 7, it returns the shared value 7.

对于 7 和 12，条件为假，函数返回 12。对于 12 和 7，第一个分支返回 12；对于 7 和 7，返回共同的值 7。

We write the choice ourselves instead of calling max because the comparison is the skill being practiced. The caller stores the returned value and then prints it.

我们自己编写选择逻辑，而不调用 max，因为本题要练习比较。调用者先保存返回值，再打印它。

Worked code and results

对照代码与运行结果

```
# Accept two integers and return the greater value.
# 接收两个整数，返回其中较大的值。
def max_of_two_numbers(first, second):
    # Choose the first value when it is greater or the values are equal.
    # 第一项更大或两项相等时，选择第一项。
    if first >= second:
        # Return the first value directly to the caller.
        # 直接把第一项交回调用处。
        return first
    # In the remaining case, the second value is strictly greater.
    # 其余情况说明第二项严格更大。
    else:
        # Return the second value so every branch returns a result.
        # 返回第二项，保证所有分支都有返回值。
        return second

# Use the first integer from the source example as editable input.
# 用题目给出的第一个整数作为可编辑输入。
first_number = 7
# Use the second integer from the source example as editable input.
# 用题目给出的第二个整数作为可编辑输入。
second_number = 12
# Call the function and save the greater value.
# 调用函数并保存较大值。
result = max_of_two_numbers(first_number, second_number)
# Display the result 12 at the call site.
# 在调用处显示结果十二。
print("Greater value:", result)

Greater value: 12
```

Worked answer

完整解答

Return the first value when it is greater than or equal to the second; otherwise return the second. The example returns 12.

当第一个值大于或等于第二个值时，返回第一个；否则返回第二个。示例返回 12。

Common mistakes

常见错误

Do not omit the equal-input case, replace return with print, or compare numeric strings instead of integers.

不要漏掉输入相等的情况，不要用 print 替换 return，也不要把数字字符串当成整数比较。

Check your understanding

检查理解

Question: If return first becomes print(first), what does the caller store for that branch?

问题：如果把 return first 改成 print(first)，调用者在该分支会保存什么？

Answer: It stores None, because printing the value does not return it.

答案：保存 None，因为打印该值不会返回它。

A choice between two values needs only two branches. Next, we will use several boundaries and keep classification separate from averaging.

两个值之间的选择只需两个分支。接下来使用多个分界，并把分类与平均数计算分开。

Exercise 5 | Test Average and Grade

练习 5｜考试平均分与等级

Write a program that asks for five test scores. Display a letter grade for each score and the average test score.

编写程序，要求输入五次考试分数。显示每个分数对应的字母等级，以及考试平均分。

1. `calc_average` accepts five test scores as arguments and returns their average.

`calc_average` 接收五个考试分数实参，并返回它们的平均数。

2. `determine_grade` accepts one test score and returns its letter grade.

`determine_grade` 接收一个考试分数，并返回字母等级。

Source omission and working assumption: the PDF mentions a grading scale but does not include it. For these worked answers, assume scores from 0 to 100 and use A for 90 or above, B for 80 to below 90, C for 70 to below 80, D for 60 to below 70, and F below 60. These thresholds are an explicit teaching assumption, not a table supplied by the PDF.

原题缺失与本解答的假设：PDF 提到评分表，却没有给出。这里假设分数在 0 到 100 之间，90 及以上为 A，80 至不足 90 为 B，70 至不足 80 为 C，60 至不足 70 为 D，低于 60 为 F。这些分界是明确声明的教学假设，不是 PDF 给出的表格。

Teaching walkthrough

逐步讲解

Our goal is to separate calculating an average from classifying a score, and to test the grade boundaries.

本题目标是把平均数计算与成绩分类分开，并测试等级边界。

The grading table is missing from the PDF, so we must state our assumption. We use scores from 0 to 100, with A at 90, B at 80, C at 70, D at 60, and F below 60.

PDF 缺失评分表，因此必须声明假设。这里使用 0 到 100 分范围，A 下界为 90，B 为 80，C 为 70，D 为 60，低于 60 为 F。

These thresholds are not a table supplied by the source. If a grading table is provided later, change the conditions in determine_grade.

这些分界不是原题提供的表格。如果之后补充了评分表，请修改 determine_grade 中的条件。

calc_average accepts five separate scores, adds them inside parentheses, and divides the total by 5. Without those parentheses, we might divide only the final score.

calc_average 接收五个独立分数，在括号内相加，再把总和除以 5。没有这些括号，就可能只把最后一个分数除以 5。

determine_grade checks the highest boundary first. A score of 95 returns A immediately, so it never reaches the B branch.

determine_grade 先检查最高分界。95 分会立即返回 A，因此不会到达 B 分支。

By the time we reach elif score >= 80, the earlier condition has already ruled out scores of 90 or above. We do not need to repeat score < 90.

到达 elif score >= 80 时，前面的条件已经排除了 90 及以上分数。因此不需要重复写 score < 90。

For 95, 82, 76, 64, and 50, the grades are A, B, C, D, and F. Their total is 367, so the average is 73.4.

对于 95、82、76、64、50，等级依次为 A、B、C、D、F。总分是 367，所以平均分是 73.4。

The task asks for each score's grade and the average score; it does not require a grade for the average. Test 90, 89.5, 80, 60, and 59.9 to check the boundaries.

题目要求每个分数的等级与平均分，不要求把平均分再转换成等级。请测试 90、89.5、80、60、59.9，核对分界。

Worked code and results

对照代码与运行结果

```
# Accept five separate test scores as required.
# 按要求接收五个独立的考试分数。
def calc_average(score1, score2, score3, score4, score5):
    # Add the five scores before dividing by five to find the average.
    # 先求总分，再除以五得到平均数。
    return (score1 + score2 + score3 + score4 + score5) / 5

# Return a letter grade using the explicitly stated assumed scale.
# 根据明确声明的暂定评分规则返回字母等级。
def determine_grade(score):
```

```
    # Start at the highest boundary: 90 or above is A.
    # 从最高分界开始，九十分及以上为 A。
    if score >= 90:
        # Returning ends the function, so later branches are not considered.
        # 返回后函数结束，不会再进入后面的分支。
        return "A"
    # Here the score is already below 90, so only the lower boundary 80 is needed.
    # 到此处已知低于九十，因此只需检查八十分下界。
    elif score >= 80:
        # Return B for scores from 80 to below 90.
        # 八十分至不足九十分返回 B。
        return "B"
    # Use this branch for scores from 70 to below 80.
    # 七十分至不足八十分使用此分支。
    elif score >= 70:
        # Return grade C.
        # 返回等级 C。
        return "C"
    # Use this branch for scores from 60 to below 70.
    # 六十分至不足七十分使用此分支。
    elif score >= 60:
        # Return grade D.
        # 返回等级 D。
        return "D"
    # Within the assumed 0-to-100 range, the remaining scores are below 60.
    # 在约定的零至一百分范围中，剩余分数低于六十。
    else:
        # Return grade F.
        # 返回等级 F。
        return "F"

# The first editable score receives A.
# 第一项可编辑成绩，对应等级 A。
score1 = 95.0
# The second editable score receives B.
# 第二项可编辑成绩，对应等级 B。
score2 = 82.0
# The third editable score receives C.
# 第三项可编辑成绩，对应等级 C。
score3 = 76.0
# The fourth editable score receives D.
# 第四项可编辑成绩，对应等级 D。
score4 = 64.0
# The fifth editable score receives F.
# 第五项可编辑成绩，对应等级 F。
score5 = 50.0
# Process all five scores in their original input order.
# 按原输入顺序逐个处理五项成绩。
for score in (score1, score2, score3, score4, score5):
    # Call the grade function and display the current score and grade.
    # 调用等级函数并显示当前成绩和等级。
    print(f"Score {score:g}: {determine_grade(score)}")
# Pass all five scores to the average function.
# 把五项成绩传入平均数函数。
average = calc_average(score1, score2, score3, score4, score5)
# Display the average 73.40 with two decimal places.
# 以两位小数显示平均分七十三点四零。
print(f"Average score: {average:.2f}")

Score 95: A
Score 82: B
Score 76: C
Score 64: D
Score 50: F
```

```
Average score: 73.40
```

Worked answer

完整解答

Using the stated assumed scale, scores 95, 82, 76, 64, and 50 have grades A, B, C, D, and F. Their average is 73.40.

使用已经声明的暂定评分表，95、82、76、64、50 分依次对应 A、B、C、D、F。平均分是 73.40。

Common mistakes

常见错误

Do not present the missing grading table as known, test low thresholds first with >=, or divide only the final score by 5.

不要把缺失评分表说成已知，不要先用 >= 检查低分界，也不要只把最后一项分数除以 5。

Check your understanding

检查理解

Question: Why is score < 90 unnecessary after elif score >= 80?

问题：为什么 elif score >= 80 后面不必再写 score < 90？

Answer: Execution reaches that branch only when the earlier score >= 90 condition is false.

答案：只有前面的 score >= 90 条件为假时，执行才会到达该分支。

We have separated two responsibilities. Now let one function reuse another function to calculate a combination.

我们已经分开两种职责。现在让一个函数复用另一个函数来计算组合数。

Exercise 6 | Combination

练习 6｜组合数

Write `combination(n, r)` to return the number of ways to choose r items from n different items when order does not matter.

编写 `combination(n, r)`，返回从 n 个不同物品中选出 r 个、不考虑顺序的选法数量。

Use `C(n, r) = n! / (r! * (n - r)!)`, where `n! = 1 * 2 * ... * n` and `0! = 1`.

使用 `C(n, r) = n! / (r! * (n - r)!)`，其中 `n! = 1 * 2 * ... * n`，且 `0! = 1`。

Requirements:

要求：

1. Write `factorial(n)` to return n!.

编写 `factorial(n)`，返回 n!。

2. Call `factorial()` inside `combination(n, r)`.

在 `combination(n, r)` 内调用 `factorial()`。

3. Assume n and r are integers and 0 <= r <= n.

假设 n 和 r 都是整数，且 0 <= r <= n。

4. Return an integer. Do not use the math module.

返回整数。不要使用 math 模块。

Examples: combination(5, 2) is 10; combination(6, 3) is 20; combination(4, 0) is 1.

示例：combination(5, 2) 为 10；combination(6, 3) 为 20；combination(4, 0) 为 1。

Teaching walkthrough

逐步讲解

Our goal is to reuse a helper function, choose the right accumulator value, and return an exact integer.

本题目标是复用辅助函数、选择正确的累积初值，并返回精确整数。

Start a product at one. A sum usually starts at zero, but starting a product at zero would make every later product zero.

乘积从 1 开始。求和通常从 0 开始，但乘积如果从 0 开始，之后每次相乘都会保持为 0。

factorial multiplies the integers from 1 through n, so the loop uses range(1, n + 1). When n is zero, the loop is empty and the initial product 1 correctly gives zero factorial.

factorial 把 1 到 n 的整数相乘，因此循环使用 range(1, n + 1)。n 为 0 时循环为空，初始乘积 1 正确给出零的阶乘。

In combination, the numerator is factorial(n), and the denominator is factorial(r) * factorial(n - r). The source guarantees integer inputs with 0 <= r <= n.

在 combination 中，分子是 factorial(n)，分母是 factorial(r) * factorial(n - r)。原题保证输入为整数且 0 <= r <= n。

For those valid inputs, the formula divides exactly. Use // so the return value is an integer rather than a floating-point result from /.

对于这些有效输入，公式可以整除。使用 //，使返回值为整数，而不是 / 得到的浮点数。

For 5 choose 2, the numerator is 120 and the denominator is 12, giving 10. For r equal to zero, the denominator is 1 times n factorial, giving 1: there is one way to choose nothing.

5 选 2 时，分子是 120、分母是 12，得到 10。r 为 0 时，分母是 1 乘 n 的阶乘，得到 1：什么都不选只有一种选法。

Worked code and results

对照代码与运行结果

```
# Calculate the factorial of non-negative n without the math module.
# 计算非负整数 n 的阶乘，不使用 math 模块。
def factorial(n):
    # Start the product at one; this also handles zero factorial.
    # 乘积从一开始，也自然覆盖零的阶乘。
    product = 1
    # range excludes its stop value, so use n plus one to include n.
    # range 不包含右端点，所以用 n 加一来包含 n。
    for number in range(1, n + 1):
```

```
        # Multiply the current integer into the running product.
        # 把当前整数乘入累积乘积。
        product *= number
    # Return the factorial after the loop; for n equal to zero, return one.
    # 循环结束后返回阶乘；n 为零时返回一。
    return product

# Calculate combinations assuming integer inputs with zero no greater than r and r no greater than n.
# 在整数且零不大于 r 不大于 n 的前提下求组合数。
def combination(n, r):
    # Reuse the factorial function to calculate the numerator n factorial.
    # 复用阶乘函数计算分子 n 的阶乘。
    numerator = factorial(n)
    # Multiply the other two factorials to form the denominator.
    # 分母由两个阶乘相乘构成。
    denominator = factorial(r) * factorial(n - r)
    # The valid formula divides exactly; double-slash division keeps the result an integer.
    # 有效组合公式能整除，双斜杠保证结果为整数。
    return numerator // denominator

# The first source example should produce 10.
# 原题第一个例子应得到十。
print(combination(5, 2))
# The second source example should produce 20.
# 原题第二个例子应得到二十。
print(combination(6, 3))
# The third source example should produce 1 and checks zero factorial.
# 原题第三个例子应得到一，检查零的阶乘。
print(combination(4, 0))
```

```
10
20
1
```

Worked answer

完整解答

Initialize the factorial product to 1. For n = 0, the loop is empty and returns 1.

阶乘的乘积从 1 开始。n 为 0 时，循环为空，返回 1。

Use integer division for the combination formula. The three source examples give 10, 20, and 1.

组合公式使用整除。原题的三个示例依次得到 10、20、1。

Common mistakes

常见错误

Do not start a product at zero, omit n from the range, return a float with /, or use math.factorial against the stated requirement.

不要把乘积从 0 开始，不要在范围中漏掉 n，不要用 / 返回浮点数，也不要违背要求使用 math.factorial。

Check your understanding

检查理解

Question: Why is factorial(0) correct even though the loop never runs?

问题：factorial(0) 从未进入循环，为什么仍然正确？

Answer: The product starts at 1 and an empty loop leaves it unchanged, matching the definition 0! = 1.

答案：乘积从 1 开始，空循环使它保持不变，符合 0! = 1 的定义。

The factorial used an integer product. Next, we will use a running sum while processing digit characters.

阶乘使用了整数乘积。接下来处理数字字符，并使用累积求和。

Exercise 7 | Sum of Digits

练习 7｜各位数字之和

Write a program that asks for a series of single-digit numbers with nothing separating them. Display the sum of all digits in the string. For example, input 2514 gives 12 because 2 + 5 + 1 + 4 = 12.

编写程序，要求输入一串没有任何分隔符的单个数字。显示字符串内所有数字之和。例如，输入 2514 得到 12，因为 2 + 5 + 1 + 4 = 12。

Assume the input contains one or more ASCII digit characters and no sign or separators.

假设输入包含一个或多个 ASCII 数字字符，没有正负号或分隔符。

Teaching walkthrough

逐步讲解

Our goal is to distinguish a string of digits from one integer and to accumulate a value character by character.

本题目标是区分数字字符串与一个整数，并逐字符累加。

For this task, keep 2514 as the string "2514". We want to visit four separate characters rather than treat the whole input as one number.

本题把 2514 保留为字符串 "2514"。我们要访问四个独立字符，而不是把整个输入当成一个数。

Start total at zero. On each iteration, convert the current character with int and add that integer to total.

把 total 从 0 开始。每轮用 int 转换当前字符，再把得到的整数加到 total。

For "2514", the running total changes from 0 to 2, then 7, then 8, and finally 12. Return the total only after the loop finishes.

对于 "2514"，累积值从 0 变成 2、7、8，最后变成 12。只有循环结束后才返回总和。

Converting the whole string to an integer first would remove the convenient character sequence. Keeping a string also preserves leading zeroes: "0012" still gives a sum of 3.

如果先把整串转成整数，就失去了方便遍历的字符序列。保留字符串还会保留前导零："0012" 的各位和仍为 3。

Worked code and results

对照代码与运行结果

```
# Accept a digit string and add its digits one at a time.
# 接收数字字符串，逐位计算数字之和。
def sum_of_digits(text):
    # Start the running sum at zero.
    # 求和累积器从零开始。
    total = 0
```

```
    # Take one digit character per iteration, rather than the whole integer.
    # 每轮取得一个数字字符，而不是整个整数。
    for character in text:
        # Convert the current character to an integer, then add it to the total.
        # 把当前字符转换成整数，再加入总和。
        total += int(character)
    # Return the digit sum after every character has been processed.
    # 所有字符处理完后返回数字和。
    return total

# Keep the source example as a string so its characters can be traversed.
# 保留原题示例为字符串，便于逐字符遍历。
digit_text = "2514"
# Call the function and display the source result 12.
# 调用函数并显示原例结果十二。
print("Sum of digits:", sum_of_digits(digit_text))
```

```
Sum of digits: 12
```

Worked answer

完整解答

Traverse the input as characters and convert each character separately. The running totals for 2514 are 2, 7, 8, and 12.

把输入作为字符序列遍历，分别转换每个字符。对于 2514，逐步累加的结果是 2、7、8、12。

Common mistakes

常见错误

Do not add digit strings directly, iterate over an integer, or return before all characters have been processed.

不要直接相加数字字符串，不要遍历一个整数，也不要在处理完所有字符之前返回。

Check your understanding

检查理解

Question: What is the digit sum of "0012"?

问题：″0012″ 的各位数字之和是多少？

Answer: It is 3. Both zeroes are visited but do not change the total.

答案：结果为 3。两个零都被访问，但不会改变总和。

We can process a string one character at a time, or split it into parts. Let us split a date at its slashes.

字符串既可以逐字符处理，也可以拆成几部分。我们按斜杠拆分一个日期。

Exercise 8 | Date Printer

练习 8｜日期打印

Read a date string in `mm/dd/yyyy` form and print it in a form such as `March 12, 2014`.

读取 `mm/dd/yyyy` 形式的日期字符串，并以 `March 12, 2014` 这样的形式打印。

Assume a valid date in the stated input format. This exercise reformats the date; it does not require calendar validation.

假设日期有效且符合指定输入格式。本题只重新组织日期表示，不要求检查日历合法性。

Teaching walkthrough

逐步讲解

Our goal is to split a structured string, use a lookup list, and format the output.

本题目标是拆分有结构的字符串、使用查找列表，并组织输出格式。

Split the input at the slashes. The three parts are the month, day, and year strings.

按斜杠拆分输入。得到的三部分依次是月份、日期和年份字符串。

The month-name list runs from January to December. Because months start at 1 but list indexes start at 0, look up month_names[month_number - 1].

月份名称列表按 January 到 December 排列。月份从 1 开始、列表索引从 0 开始，因此查找 month_names[month_number - 1]。

Convert the day to an integer before displaying it. An input day of "02" will then appear as 2, while the four-digit year can remain a string.

显示日期之前，把日数转换成整数。输入 "02" 就会显示成 2，而四位年份可以保留为字符串。

We assume a valid date in the stated format. This function reformats a date; it does not establish that a date such as February 31 exists.

这里假设日期有效且符合指定格式。函数只是重新组织日期表示，不会证明 2 月 31 日这样的日期真实存在。

Worked code and results

对照代码与运行结果

```
# Accept a valid month/day/year string and print an English date.
# 接收有效的月日年字符串，输出英文日期。
def print_date(date_text):
    # Create a month-name lookup list in January-to-December order.
    # 按一月至十二月顺序建立月份名称表。
    month_names = [
        "January", "February", "March", "April", "May", "June",
        "July", "August", "September", "October", "November", "December"
    ]
    # Split at each slash to obtain the month, day, and year strings.
    # 用斜杠把输入拆成月、日、年三个字符串。
    parts = date_text.split("/")
    # Convert the first part to an integer month number.
    # 把第一部分转换成整数月份。
    month_number = int(parts[0])
    # Convert the day to an integer so its leading zero is not displayed.
    # 转成整数日数，显示时去掉前导零。
    day = int(parts[1])
    # Keep the third part as the four-character year string.
    # 第三部分保留为四位年份字符串。
    year = parts[2]
    # Months start at one, but list indexes start at zero.
    # 月份从一开始，列表索引从零开始。
    month_name = month_names[month_number - 1]
    # Print the English month name, day, and year in the requested order.
    # 按英文月份名、日和年的顺序输出。
    print(f"{month_name} {day}, {year}")
```

```
# Use a valid example corresponding to the source's requested output.
# 用与原题目标输出对应的有效示例日期。
date_text = "03/12/2014"
# Call the date-printing function to display March 12, 2014.
# 调用日期打印函数，显示 March 12, 2014。
print_date(date_text)
```

```
March 12, 2014
```

Worked answer

完整解答

Split on the slash. Subtract 1 from the numeric month to index the month-name list. The example prints March 12, 2014.

按斜杠拆分。把数字月份减 1，作为月份名称列表的索引。示例打印 March 12, 2014。

Common mistakes

常见错误

Do not forget to subtract one for the month index, swap month and day, or describe reformatting as full date validation.

不要忘记月份索引减 1，不要交换月与日，也不要把格式转换说成完整日期验证。

Check your understanding

检查理解

Question: Which list index corresponds to month 12?

问题：月份 12 对应列表的哪个索引？

Answer: Index 11, because indexes begin at zero.

答案：索引 11，因为索引从零开始。

A date has fixed separators. A sentence needs a small state variable to remember where its beginning is.

日期有固定分隔符。句子则需要一个小型状态变量来记住句首位置。

Exercise 9 | Sentence Capitalizer

练习 9｜句首大写

Write a function that accepts a string and returns a copy with the first character of each sentence capitalized. For example,

编写函数，接收字符串并返回一个副本，使每个句子的首字符大写。例如，

```
hello my name is Joe. what is your name?
```

becomes

变为

```
Hello my name is Joe. What is your name?
```

Let the user enter a string, pass it to the function, and display the modified string.

让用户输入字符串，传给函数，再显示修改后的字符串。

Working rule: . , ?, and ！ mark sentence boundaries. Capitalize the first alphabetic character at the beginning or after one of those boundaries, preserving spaces, punctuation, and every other character's original case.

采用的规则：. 、?、！ 表示句子边界。把文本开头或边界之后的第一个字母大写，保留空格、标点，以及其他每个字符原有的大小写。

Teaching walkthrough

逐步讲解

Our goal is to track a sentence beginning with a Boolean and build a new string without changing the other text.

本题目标是用布尔值跟踪句首，并在不改变其他文本的情况下构造新字符串。

Remember whether the next letter starts a sentence. Set needs_capital to True at the beginning, and start result as an empty string.

记住下一个字母是否位于句首。开始时把 needs_capital 设为 True，把 result 设为空字符串。

When needs_capital is true and the current character is alphabetic, uppercase that character and change the state to False. The remaining letters in that sentence keep their original case.

当 needs_capital 为真且当前字符是字母时，把该字符大写，并把状态改成 False。该句其余字母保留原有大小写。

When the current character is a period, question mark, or exclamation mark, set the state back to True. Spaces and other punctuation leave that waiting state unchanged.

当前字符为句号、问号或感叹号时，把状态重新设为 True。空格和其他标点不会改变这个等待状态。

Do not use capitalize on the whole text, because it can lowercase later letters and change Joe to joe. title would capitalize every word, which is a different task.

不要对整个文本使用 capitalize，因为它可能把后续字母变小写，把 Joe 改成 joe。title 会把每个单词首字母大写，那是另一项任务。

The result is Hello my name is Joe. What is your name? Only the intended sentence beginnings change.

结果是 Hello my name is Joe. What is your name? 只有约定的句首发生变化。

Strings are immutable, so the function returns a new result string. The caller should use that returned value rather than assume the original input was changed in place.

字符串不可变，因此函数返回新的结果字符串。调用者应使用这个返回值，不要假设原输入已被原地修改。

Worked code and results

对照代码与运行结果

```
# Return a new string using the explicitly stated sentence-boundary rule.
# 根据明确约定的句界规则返回新字符串。
def capitalize_sentences(text):
    # Start with an empty string for the result being built.
    # 用空字符串保存逐步构造的结果。
    result = ""
```

```
    # At the start of the text, treat the next letter as a sentence beginning.
    # 文本开始时，下一个字母视为句首。
    needs_capital = True
    # Process each character in its original order.
    # 保留原顺序逐个处理字符。
    for character in text:
        # Capitalize only when waiting for a sentence beginning and the character is a letter.
        # 只有等待句首且遇到字母时才大写。
        if needs_capital and character.isalpha():
            # Convert the current sentence's first letter to uppercase.
            # 把当前句首字母转换为大写。
            character = character.upper()
            # The sentence beginning is handled; preserve the remaining letters.
            # 本句句首已处理，其他字母保持原样。
            needs_capital = False
        # Prepare for another sentence after one of the stated ending marks.
        # 遇到约定的句末标点后准备处理下一句。
        if character in ".?!":
            # Spaces and other punctuation do not cancel the waiting state.
            # 空格和其他标点不会取消这个等待状态。
            needs_capital = True
        # Append the current character, preserving the other case choices and whitespace.
        # 把当前字符加入新字符串，保留其他大小写和空白。
        result += character
    # Return a new string without modifying the original input string.
    # 返回新字符串，而不修改原输入字符串。
    return result

# Keep the source's complete original example.
# 保留题目给出的完整原始示例。
text = "hello my name is Joe. what is your name?"
# Display the capitalized sentence beginnings while preserving Joe.
# 显示句首大写且 Joe 保持原样的结果。
print(capitalize_sentences(text))
```

```
Hello my name is Joe. What is your name?
```

Worked answer

完整解答

A Boolean remembers whether the next alphabetic character starts a sentence. Preserve every other character, including the capital J in Joe. The result is Hello my name is Joe. What is your name?

用布尔值记住下一个字母是否位于句首。其他字符都保留，包括 Joe 中的大写 J。结果是 Hello my name is Joe. What is your name?

Common mistakes

常见错误

Do not lowercase Joe with capitalize, capitalize every word with title, or cancel the waiting state when a space follows punctuation.

不要用 capitalize 把 Joe 变小写，不要用 title 把每个词首字母大写，也不要在标点后的空格处取消等待状态。

Check your understanding

检查理解

Question: Why should needs_capital remain True for the space after a period?

问题：为什么句号后的空格不应让 needs_capital 变成 False？

Answer: We have not reached the next sentence's first letter yet; the space is only a character to preserve.

答案：我们还没有遇到下一句的第一个字母；空格只是需要保留的字符。

We just built a new string character by character. Now keep the text unchanged and calculate two summaries of it.

刚才我们逐字符构造了新字符串。现在保持文本不变，计算它的两项统计信息。

Exercise 10 | Most Frequent Character

练习 10｜最频繁字符

Write a program that lets the user enter a string and displays its most frequent character. Add a function that returns the number of vowels in that string. Hint: use the string method `count()` if helpful.

编写程序，让用户输入字符串并显示出现最频繁的字符。再添加一个函数，返回字符串中的元音数量。提示：可以使用字符串方法 `count()`。

Working rules: count character frequencies case-sensitively, including spaces and punctuation; on a tie, choose the character that appears first in the string. For empty input, report no most frequent character. Count English vowels `a`, `e`, `i`, `o`, `u` in either case.

采用的规则：字符频率区分大小写，包含空格和标点；次数相同时，选择字符串里最先出现的字符。空输入没有最频繁字符。英文元音 `a`、`e`、`i`、`o`、`u` 的大小写都计数。

Teaching walkthrough

逐步讲解

Our goal is to find a highest frequency and count vowels, while making tie and empty-input rules explicit.

本题目标是寻找最高频率并统计元音，同时明确平局和空输入规则。

A tie rule is part of the specification. Here frequency counting is case-sensitive and includes spaces and punctuation; ties keep the character that appears first.

平局规则也是题意的一部分。这里的频率统计区分大小写，包含空格和标点；次数相同时保留最先出现的字符。

Return None immediately for an empty string, because there is no first character to inspect. For non-empty input, use the first character as an initial candidate and start best_count at zero.

空字符串立即返回 `None`，因为没有第一个字符可供检查。非空输入先把首字符作为候选，并把 `best_count` 从 `0` 开始。

For each character, text.count(character) finds its total frequency in the input. Update the best choice only when current_count > best_count, so an equal count cannot replace an earlier character.

对每个字符，`text.count(character)` 求出它在输入中的总次数。只有 `current_count > best_count` 时才更新，因此相同次数不会替换更早的字符。

For vowels, we use a different stated rule: count a, e, i, o, and u in either case. Convert the text to lowercase, then add one whenever a character belongs to "aeiou".

元音使用另一条明确规则：`aeiou` 的大小写都计数。先把文本转换成小写，再在字符属于 `"aeiou"` 时加 `1`。

In "Hello World", lowercase l occurs three times, and the vowels are e, o, and o, also three. Using repr for the character makes a space answer visible as ' '.

在 "Hello World" 中，小写 l 出现三次；元音为 e、o、o，也是三个。对字符使用 repr，可以让空格答案显示为 ' '。

Worked code and results

对照代码与运行结果

```
# Return the most frequent character, keeping the earliest character on a tie.
# 返回最频繁字符，平局时保留最早出现者。
def most_frequent_character(text):
    # An empty string has no characters, so handle it first.
    # 空字符串没有字符，必须先单独处理。
    if text == "":
        # Use None to represent the absence of a most frequent character.
        # 用 None 表示不存在最频繁字符。
        return None
    # For non-empty input, the first character is a safe initial candidate.
    # 非空时可以安全地把首字符作为暂时答案。
    best_character = text[0]
    # Start the highest frequency at zero.
    # 暂时最大次数从零开始。
    best_count = 0
    # Examine characters in input order to preserve the tie preference.
    # 按原字符串顺序考察字符，保留平局优先顺序。
    for character in text:
        # Use the hinted string method to count this character's total occurrences.
        # 使用题目提示的方法统计该字符总次数。
        current_count = text.count(character)
        # Update only for a strictly larger count; do not replace an earlier character on a tie.
        # 仅严格更多时更新，相等时不覆盖较早字符。
        if current_count > best_count:
            # Save the new most frequent character.
            # 保存新的最高频字符。
            best_character = character
            # Also save its occurrence count.
            # 同时保存它的出现次数。
            best_count = current_count
    # Return the character itself after the traversal.
    # 遍历结束后返回字符本身。
    return best_character

# Count English vowels in either uppercase or lowercase.
# 统计英文元音，大小写都包括在内。
def count_vowels(text):
    # Start the vowel count at zero.
    # 元音计数从零开始。
    total = 0
    # Convert to lowercase first so one set of vowel characters is enough.
    # 先统一小写，便于使用一组元音字符判断。
    for character in text.lower():
        # Count only the five stated English vowels.
        # 只有五个约定的英文元音才计数。
        if character in "aeiou":
            # Increase the count by one for each vowel.
            # 每遇到一个元音就把计数增加一。
            total += 1
    # Return the total instead of printing inside the function.
    # 返回元音总数，而不是在函数内打印。
    return total
```

```
# Use an editable example containing uppercase and lowercase letters and a space.
# 可编辑示例包含大小写字母和一个空格。
text = "Hello World"
# Call the frequency function to obtain a character or None.
# 调用频率函数取得答案或 None。
character = most_frequent_character(text)
# Check the special return value for empty input first.
# 先判断空字符串对应的特殊返回值。
if character is None:
    # Display a clear English message for empty input.
    # 为空输入显示明确的英文提示。
    print("No most frequent character.")
# For non-empty input, display the returned character.
# 非空输入时可以显示得到的字符。
else:
    # repr makes spaces and similar characters visible in the output.
    # repr 让空格等字符也能清楚显示。
    print("Most frequent character:", repr(character))
# Call the vowel function and display the count.
# 调用元音函数并显示计数。
print("Vowels:", count_vowels(text))
```

```
Most frequent character: 'l'
Vowels: 3
```

Worked answer

完整解答

Use a strict greater-than comparison so ties keep the first character encountered. For Hello World, the most frequent character is lowercase l, and the vowel count is 3.

使用严格大于比较，使次数相同时保留最先遇到的字符。对于 Hello World，出现最频繁的是小写 l，元音数量为 3。

Common mistakes

常见错误

Do not use >= if ties should keep the first character, index an empty string, miss uppercase vowels, or silently ignore spaces.

平局要保留首个字符时不要用 >=；不要索引空字符串，不要漏掉大写元音，也不要未经说明忽略空格。

Check your understanding

检查理解

Question: For "ba", which character wins the frequency tie under our rule?

问题：对于 "ba"，按照本讲义规则，哪个字符在频率平局时胜出？

Answer: b wins because it appears first. The later a does not pass the strict greater-than test.

答案：b 胜出，因为它先出现。后面的 a 不会通过严格大于判断。

We have practiced loops, conditions, and helper functions separately. Next, we will combine them and check zero and empty results carefully.

我们已经分别练习循环、条件和辅助函数。接下来把它们组合起来，并仔细检查零与空结果。

Exercise 11 | Armstrong Numbers

练习 11 | Armstrong 数

Find Armstrong numbers in a range and calculate their count, sum, and mean. Use integers, loops, and functions.

找出一个范围中的 Armstrong 数，计算数量、总和及平均数。使用整数、循环和函数。

A non-negative integer with k digits is an Armstrong number if it equals the sum of its digits raised to the power k. Treat 0 as a one-digit number.

一个有 k 位的非负整数，如果等于各位数字 k 次方之和，就是 Armstrong 数。把 0 视为一位数。

Examples: 153 = 1 ** 3 + 5 ** 3 + 3 ** 3; 1634 = 1 ** 4 + 6 ** 4 + 3 ** 4 + 4 ** 4.

示例：153 = 1 ** 3 + 5 ** 3 + 3 ** 3；1634 = 1 ** 4 + 6 ** 4 + 3 ** 4 + 4 ** 4。

Complete the functions in this framework, in order:

按顺序补全下列框架中的函数：

```
def count_digits(n):
    # TODO: Count digits using a while loop and // 10.
    pass

def digit_power_sum(n):
    # TODO: Extract digits and add each digit ** k.
    pass

def is_armstrong(n):
    # TODO: Check whether n is an armstrong. Return a boolean.

def report_armstrong(start, end):
    # TODO:
    # 1. Show all armstrong between start and end
    # 2. Calculate and print the mean to 2 decimals.
    pass

def main():
    start = int(input("Enter start: "))
    end = int(input("Enter end: "))
    Call report_armstrong().

main()
```

Source note: `Call report_armstrong().` is an instruction, not valid Python. Replace it with `report_armstrong(start, end)`. Include count and sum as required by the opening prompt, even though the framework's TODO mentions only display and mean.

原题说明：`Call report_armstrong().` 是说明语句，不是有效 Python。把它替换成 `report_armstrong(start, end)`。题目开头要求数量和总和，即使框架 TODO 只写显示与平均数，也要包含这两项。

Working rules: use a non-negative inclusive range with `start <= end`. For no matches, display count 0, sum 0, and `Mean: N/A`. The runnable example uses editable values 0 and 500 instead of blocking input.

采用的规则：使用非负且包含两端的范围，满足 `start <= end`。没有匹配时，显示数量 0、总和 0、`Mean: N/A`。可运行示例使用可编辑数值 0 和 500，避免等待输入。

Teaching walkthrough

逐步讲解

Our goal is to build small numeric functions first, then combine them into a complete report.

本题目标是先建立小型数值函数，再把它们组合成完整报告。

count_digits handles only the digit count. digit_power_sum uses that result, is_armstrong compares the sum with the original number, and report_armstrong handles the range.

count_digits 只处理位数。digit_power_sum 使用这个结果，is_armstrong 比较数位幂和与原数，report_armstrong 处理整个范围。

The source requires a while loop and // 10 for counting digits. Handle n == 0 first by returning 1, because zero is explicitly a one-digit number.

原题要求用 while 循环和 // 10 统计位数。先处理 n == 0，返回 1，因为题目明确规定零是一位数。

For 153, repeatedly dividing by ten gives 153, 15, 1, and 0, so the loop counts three digits. Changing this local integer parameter does not change the caller's integer variable.

对于 153，重复整除十得到 153、15、1、0，因此循环统计到三位。修改这个局部整数形参，不会修改调用者的整数变量。

In digit_power_sum, save k before removing any digits. Each iteration uses n % 10 for the last digit, adds digit ** k, and uses n //= 10 to remove that digit.

在 digit_power_sum 中，去掉任何数字之前先保存 k。每轮用 n % 10 得到末位，累加 digit ** k，再用 n //= 10 去掉该位。

For 153, we process 3, 5, and 1, giving 27 + 125 + 1 = 153. For zero, the loop does not run and the total stays 0, which is also 0 ** 1.

对于 153，处理顺序是 3、5、1，得到 27 + 125 + 1 = 153。对于零，循环不运行，总和保持 0，也正好等于 0 ** 1。

is_armstrong returns the Boolean comparison itself. It should not merely print True or False, because the report function needs that returned value in an if condition.

is_armstrong 直接返回布尔比较结果。它不能只是打印 True 或 False，因为报告函数需要把返回值用于 if 条件。

Use range(start, end + 1) to include the upper endpoint. Whenever a number matches, print it, add one to count, and add the number to total.

使用 range(start, end + 1) 包含上端点。每发现一个匹配，就打印它、把 count 加 1，并把该数加到 total。

After the loop, display count and sum as well as mean. Calculate total / count only when count is positive, and display two decimal places; otherwise show Mean: N/A.

循环后显示数量、总和与平均数。只有 count 为正时才计算 total / count，并显示两位小数；否则显示 Mean: N/A。

From 0 through 500, the matches are 0 through 9, followed by 153, 370, 371, and 407. There are 14 values with sum 1346 and mean 96.14; the source example 1634 is outside this range but still satisfies the test.

从 0 到 500，匹配数是 0 到 9，再加上 153、370、371、407。共有 14 个数，总和 1346，平均数 96.14；原题示例 1634 在此范围外，但仍满足判断。

Finally, main chooses the editable bounds and calls report_armstrong(start, end). Replace the source's English placeholder with that real call; for interaction, the bounds can come from int(input(...)).

最后由 main 选择可编辑上下界，并调用 report_armstrong(start, end)。把原题英文占位句换成这个真正调用；交互时可通过 int(input(...)) 取得上下界。

Worked code and results

对照代码与运行结果

```
# Count the decimal digits of a non-negative integer using an integer loop.
# 用整数循环计算非负整数的十进制位数。
def count_digits(n):
    # Handle zero separately; otherwise the following loop would not run.
    # 零需要单独处理，否则下面的循环一次也不执行。
    if n == 0:
        # The source explicitly treats zero as a one-digit number.
        # 题目明确规定零是一位数。
        return 1
    # For a nonzero integer, start the number of counted digits at zero.
    # 对非零整数，已经统计的位数从零开始。
    count = 0
    # Continue while there are digits left to remove.
    # 只要还有未去掉的数字，就继续统计。
    while n > 0:
        # Count one digit on each iteration.
        # 每次循环对应当前的一位。
        count += 1
        # Integer division by ten removes the final digit and eventually reaches zero.
        # 整除十去掉末位，逐步让整数变成零。
        n //= 10
    # Return the number of digits found by the loop.
    # 返回完成循环后得到的位数。
    return count

# Calculate the sum of each digit raised to the power k.
# 计算各位数字的 k 次方之和。
def digit_power_sum(n):
    # Find the original digit count before removing any digits.
    # 先确定原整数位数，不能等去位后再计算。
    k = count_digits(n)
    # Start the sum of digit powers at zero.
    # 各位幂的求和从零开始。
    total = 0
    # Process the current final digit on each iteration.
    # 每轮处理当前整数的末位数字。
    while n > 0:
        # The remainder modulo ten is the final digit.
        # 除十余数就是当前末位。
        digit = n % 10
        # Add the current digit raised to k to the running total.
        # 将末位的 k 次方加入总和。
        total += digit ** k
        # Remove the processed final digit before the next iteration.
        # 去掉处理过的末位，准备下一轮。
        n //= 10
    # For zero, the loop is skipped and the function returns zero, which equals zero to the first power.
    # 零不进入循环而返回零，也等于零的一次方。
    return total

# Express the Armstrong-number definition with one Boolean comparison.
# 用一个布尔比较表达 Armstrong 数的定义。
def is_armstrong(n):
```

```
    # Return True when the original equals its digit-power sum, otherwise False.
    # 原数等于各位幂之和时返回 True，否则 False。
    return n == digit_power_sum(n)

# Find and summarize matches in a non-negative range including both endpoints.
# 在包含两端的非负区间中寻找并汇总结果。
def report_armstrong(start, end):
    # Start the number of matches at zero.
    # 匹配数量从零开始。
    count = 0
    # Start the sum of matching integers at zero.
    # 匹配整数总和从零开始。
    total = 0
    # Print a heading for the numbers that follow on separate lines.
    # 打印结果标题，使后面的逐行数字含义明确。
    print("Armstrong numbers:")
    # Add one to the stop value so end itself is checked.
    # 结束位置加一，确保把 end 本身也检查。
    for number in range(start, end + 1):
        # Call the predicate and process only numbers satisfying the definition.
        # 调用判断函数，仅处理满足定义的整数。
        if is_armstrong(number):
            # Display the current match in increasing order.
            # 按从小到大的顺序显示当前匹配数。
            print(number)
            # Increase the count for every match found.
            # 每发现一个匹配就增加数量。
            count += 1
            # Also add that number to the running sum.
            # 同时将这个数加入统计总和。
            total += number
    # Display the count even when it remains zero.
    # 即使没有匹配也显示数量零。
    print("Count:", count)
    # Display the sum, which remains zero when there are no matches.
    # 显示匹配数总和，无匹配时保持零。
    print("Sum:", total)
    # The mean is defined only when the count is nonzero.
    # 只有数量非零时平均数才有定义。
    if count > 0:
        # Divide the sum by the number of matches to calculate the mean.
        # 用总和除以匹配数量计算平均数。
        mean = total / count
        # Display two decimal places as required.
        # 按题目要求保留两位小数。
        print(f"Mean: {mean:.2f}")
    # With no matches, do not divide by zero.
    # 没有匹配时不能执行除以零。
    else:
        # Use a clear English marker to indicate an undefined mean.
        # 用清楚的英文标记表示平均数不存在。
        print("Mean: N/A")

# Keep the framework's main function to organize the inputs and report call.
# 保留原框架的主程序结构，组织输入和报告调用。
def main():
    # Use an editable lower bound; replace it with int(input("Enter start: ")) for interaction.
    # 可编辑下界；交互演示时可换成 int(input("Enter start: "))。
    start = 0
    # Use an editable upper bound; replace it with int(input("Enter end: ")) for interaction.
    # 可编辑上界；交互演示时可换成 int(input("Enter end: "))。
    end = 500
```

```
    # Replace the framework's English instruction with an actual function call.
    # 使用真正的函数调用替换框架中的英文说明。
    report_armstrong(start, end)

# Call main so the full example runs directly.
# 调用主程序，使整段示例可以直接运行。
main()
```

```
Armstrong numbers:
0
1
2
3
4
5
6
7
8
9
153
370
371
407
Count: 14
Sum: 1346
Mean: 96.14
```

Worked answer

完整解答

Use % 10 to extract a digit and // 10 to remove it. Handle zero as one digit.

使用 % 10 提取末位，使用 // 10 去掉末位。把 0 作为一位数处理。

On the inclusive range 0 to 500, there are 14 matches with sum 1346 and mean 96.14. On 10 to 99, print Count: 0, Sum: 0, and Mean: N/A.

在包含两端的 0 至 500 范围内，有 14 个匹配，总和 1346，平均数 96.14。在 10 至 99 范围内，打印 Count: 0、Sum: 0、Mean: N/A。

Common mistakes

常见错误

Do not count zero as having no digits, find k after removing digits, use / to remove a digit, omit end, forget count or sum, or divide by zero for an empty result.

不要把零当作没有数位，不要去位后才求 k，不要用 / 去位，不要漏掉 end，不要漏报数量或总和，也不要在空结果时除以零。

Check your understanding

检查理解

Question: What should the report show for the range 10 through 99, which has no matches?

问题：10 到 99 没有匹配数时，报告应显示什么？

Answer: Show Count: 0, Sum: 0, and Mean: N/A. The mean is undefined, so 0.00 would not be a valid mean.

答案：显示 Count: 0、Sum: 0、Mean: N/A。平均数没有定义，因此 0.00 不能作为有效平均数。

The Armstrong report combines functions by calculation level. Our last program combines functions in validation order, moving on only after a check passes.

Armstrong 报告按计算层次组合函数。最后的程序按验证顺序组合函数，只有检查通过后才继续。

Exercise 12 | Chinese ID number check

练习 12｜中国居民身份证号检查

Check the format and checksum of an 18-character Chinese resident ID string, then extract the birth date and sex code. Complete all functions and TODOs, adding lines as needed.

检查一个 18 字符居民身份证号字符串的格式和校验和，再提取出生日期与性别码。补全所有函数和 TODO，必要时添加代码行。

1. `has_valid_format(id_number)`: require exactly 18 characters. The first 17 must be ASCII digits 0 to 9. The final character must be a digit or X.

`has_valid_format(id_number)`：必须恰好 18 字符。前 17 个必须是 ASCII 数字 0 到 9。最后一位必须是数字或 X。

2. `calculate_check_digit(first17)`: assume exactly 17 digits. For each index i from 0 through 16, compute `weight = (2 ** (17 - i)) % 11`; add `int(first17[i]) * weight` to a running total.

`calculate_check_digit(first17)`：假设输入恰好 17 个数字。对索引 i 从 0 到 16，计算 `weight = (2 ** (17 - i)) % 11`，把 `int(first17[i]) * weight` 加入累积总和。

Set `remainder = total % 11`. Return the character X for remainder 10, otherwise return the remainder as a digit character.

令 `remainder = total % 11`。余数为 10 时返回字符 X，否则返回对应的数字字符。

3. `has_valid_checksum(id_number)`: call the checksum function on the first 17 characters and compare with the final character.

`has_valid_checksum(id_number)`：对前 17 个字符调用校验函数，并与最后一个字符比较。

4. `get_birth_date(id_number)`: use indexes 6 through 13, containing YYYYMMDD, to return YYYY-MM-DD. `get_sex(id_number)`: the digit at index 16 is odd for Male and even for Female.

`get_birth_date(id_number)`：索引 6 到 13 包含 YYYYMMDD，用来返回 YYYY-MM-DD。`get_sex(id_number)`：索引 16 的数字为奇数时返回 Male，偶数时返回 Female。

5. `main()`: read a string, strip surrounding spaces, and convert x to X. Check format first and checksum second.

`main()`：读取字符串、去掉外围空格，并把 x 转成 X。先检查格式，再检查校验和。

Print an error and stop if either check fails. Otherwise display the success line, birth date, and sex.

任一检查失败时打印错误并停止。否则显示成功提示、出生日期和性别。

Source example:

原题样例：

```
Input: 00000020000101038X
Format and checksum passed.
Date of birth: 2000-01-01
Sex: Female
```

Source inconsistency: the stated checksum formula gives a weighted total of 46 and remainder 2 for the first 17 characters of this sample. Therefore its expected check character is 2, not X.

原题不一致：按题目给出的校验公式，这个样例前 17 个字符的加权总和为 46、余数为 2。因此预期校验字符是 2，不是 X。

The original sample fails the stated checksum rule. The separately corrected classroom-rule example is 000000200001010382; it produces the three success lines above.

原样例无法通过其指定的校验规则。另列的课堂规则修正样例是 000000200001010382，它会产生上面的三行成功输出。

Original framework:

原题框架：

```
def has_valid_format(id_number):
    pass

def calculate_check_digit(first17):
    pass

def has_valid_checksum(id_number):
    pass

def get_birth_date(id_number):
    pass

def get_sex(id_number):
    pass

def main():
    id_number = input("Enter an ID number: ").strip().upper()
    # TODO: Check format; print an error and return if needed.
    # TODO: Check checksum; print an error and return if needed.
    print("Format and checksum passed.")
    print("Date of birth:", get_birth_date(id_number))
    print("Sex:", get_sex(id_number))

main()
```

Use exactly the classroom checksum rule stated above. This exercise checks only the stated format and checksum and extracts fields; it does not validate real identity, region codes, or calendar dates. The runnable main accepts an editable sample string instead of waiting for input.

严格使用上面给出的课堂校验规则。本题只检查指定格式与校验和并提取字段，不验证真实身份、地区码或日历日期。可运行主函数接收可编辑示例字符串，不等待输入。

Teaching walkthrough

逐步讲解

Our goal is to validate a string in stages, then extract information only after the required checks pass.

本题目标是分阶段验证字符串，在必要检查通过后才提取信息。

Keep the ID as a string because it may have leading zeroes and may end in X. Validate the length before using fixed indexes.

把号码保留为字符串，因为它可能有前导零，末位也可能是 X。使用固定索引之前先验证长度。

has_valid_format requires exactly 18 characters, then checks the first 17 against "0123456789". The final position allows a digit or uppercase X.

has_valid_format 要求恰好 18 个字符，再对照 "0123456789" 检查前 17 位。最后一位允许数字或大写 X。

A bare isdigit check is broader than the source's rule because it accepts some non-ASCII numerals. The explicit digit string keeps the check limited to 0 through 9.

单独使用 isdigit 比原题规则更宽，因为它接受某些非 ASCII 数字。显式数字字符串把检查限制在 0 到 9。

calculate_check_digit visits indexes 0 through 16. For each position, calculate weight = (2 ** (17 - i)) % 11, multiply by that digit, and add the product to total.

calculate_check_digit 访问索引 0 到 16。每个位置计算 weight = (2 ** (17 - i)) % 11，乘以该位数字，再把乘积加到 total。

After the loop, take total % 11. Return "X" when the remainder is 10; otherwise return str(remainder), because the final ID character is a string too.

循环后计算 total % 11。余数为 10 时返回 "X"，否则返回 str(remainder)，因为号码末位也是字符串。

has_valid_checksum uses [:17] to calculate the expected character and compares it with [-1]. In main, this function is called only after the format passes.

has_valid_checksum 使用 [:17] 计算预期字符，并与 [-1] 比较。在 main 中，这个函数只在格式通过后调用。

The date slices are [6:10] for the year, [10:12] for the month, and [12:14] for the day. Remember that a slice excludes its right endpoint.

日期切片是年份 [6:10]、月份 [10:12]、日数 [12:14]。注意切片不包括右端点。

Convert the digit at index 16 to an integer to check parity. An odd digit returns Male, and an even digit returns Female.

把索引 16 的数字转换成整数，检查奇偶。奇数返回 Male，偶数返回 Female。

main first applies strip().upper(). It checks format, returns early on failure, checks the checksum next, and again returns early on failure; only then does it display the extracted fields.

main 先执行 strip().upper()。它检查格式，失败就提前返回；然后检查校验和，失败再次提前返回；最后才显示提取的字段。

The PDF's sample contains an inconsistency. For the first 17 characters of 00000020000101038X, the weighted total is 46 and the remainder is 2, so the stated rule requires a final 2 rather than X.

PDF 样例存在不一致。对于 00000020000101038X 的前 17 位，加权总和为 46、余数为 2，因此给定规则要求末位为 2，而不是 X。

Preserve that original sample and report Invalid checksum. Then show the separately labeled classroom-rule correction, 000000200001010382, which passes and yields 2000-01-01 and Female.

保留这个原样例，并报告 Invalid checksum.。然后展示另行标注的课堂规则修正样例 000000200001010382，它可以通过，并提取出 2000-01-01 和 Female。

Do not change the formula to force the source example to pass. This exercise checks only its stated format and checksum; it does not establish a real identity, validate region codes, or check calendar dates.

不要为了让原样例通过而修改公式。本题只检查它规定的格式和校验和，不验证真实身份、地区码或日历日期。

Worked code and results

对照代码与运行结果

```
# Check the length and the ASCII character range required by the source.
# 检查长度以及题目允许的 ASCII 字符范围。
def has_valid_format(id_number):
    # Check the length first to prevent later index errors.
    # 长度必须先检查，避免后续索引越界。
    if len(id_number) != 18:
        # Reject the format immediately if the length is wrong.
        # 长度错误时立即报告格式不通过。
        return False
    # Examine the first seventeen characters in order.
    # 依次检查前十七个字符。
    for character in id_number[:17]:
        # Require zero through nine explicitly, excluding other Unicode numerals.
        # 明确限定零至九，排除其他 Unicode 数字。
        if character not in "0123456789":
            # Stop checking as soon as an invalid character is found.
            # 发现一个非法字符就无需继续检查。
            return False
    # Allow a digit character or uppercase X in the final position.
    # 最后一位允许数字字符或大写 X。
    return id_number[-1] in "0123456789X"

# Assume seventeen valid digit characters and calculate the classroom check digit.
# 假设已收到十七个有效数字字符，计算课堂校验位。
def calculate_check_digit(first17):
    # Start the weighted sum at zero.
    # 加权乘积总和从零开始。
    total = 0
    # Use indexes zero through sixteen, covering all seventeen positions.
    # 依次使用索引零到十六，共十七个位置。
    for i in range(17):
        # Follow exactly the power-and-remainder weight formula given in the source.
        # 严格按原题给定的幂和取余公式计算权重。
        weight = (2 ** (17 - i)) % 11
        # Multiply the current digit by its weight and add the product to the total.
        # 将当前位置数字乘权重，再加入总和。
        total += int(first17[i]) * weight
    # Find the final remainder only after all positions have been accumulated.
    # 所有位置累加完成后才计算最终余数。
    remainder = total % 11
    # Ten is not a single digit character, so the source represents it as X.
    # 十不能用单个数字字符表示，题目规定改用 X。
    if remainder == 10:
        # Return an uppercase string character for the later character comparison.
        # 返回大写字符串字符，方便之后逐字符比较。
        return "X"
    # Convert every other remainder to a string too, rather than returning an integer.
    # 其他余数也必须转成字符串，而不是返回整数。
    return str(remainder)

# The main program calls this function only after the format has passed.
# 主程序在格式通过后才调用此函数。
def has_valid_checksum(id_number):
    # Calculate the expected check character from the first seventeen positions.
    # 取前十七位计算预期校验字符。
    expected = calculate_check_digit(id_number[:17])
    # Compare it with the actual final character and return a Boolean.
    # 与实际末位比较并返回布尔结果。
    return expected == id_number[-1]
```

```
# Extract the birth-date string from the fixed positions given in the source.
# 按题目给出的固定位置提取出生日期字符串。
def get_birth_date(id_number):
    # Indexes six through nine contain the four-digit year.
    # 索引六至九共四位表示年份。
    year = id_number[6:10]
    # Indexes ten and eleven contain the two-digit month.
    # 索引十至十一共两位表示月份。
    month = id_number[10:12]
    # Indexes twelve and thirteen contain the two-digit day.
    # 索引十二至十三共两位表示日期。
    day = id_number[12:14]
    # Join the three parts with hyphens in the requested format.
    # 使用连字符把三段组合成指定格式。
    return f"{year}-{month}-{day}"

# Use the parity of the digit at index sixteen to return an English sex code.
# 使用索引十六的数字奇偶性返回英文性别码。
def get_sex(id_number):
    # Convert the stated character to an integer before checking its parity.
    # 把指定字符转成整数，才能进行奇偶判断。
    sex_code = int(id_number[16])
    # A remainder of one on division by two means the digit is odd.
    # 除二余一说明该位是奇数。
    if sex_code % 2 == 1:
        # Under the source's rule, an odd digit corresponds to Male.
        # 按题目规则，奇数对应 Male。
        return "Male"
    # The format guarantees a digit, so the remaining even case corresponds to Female.
    # 格式已保证是数字，剩余偶数对应 Female。
    return "Female"

# Use an editable string parameter instead of blocking input while keeping main's role.
# 用可编辑字符串参数代替阻塞输入，同时保留主程序职责。
def main(raw_id):
    # Remove surrounding spaces and normalize lowercase x to uppercase.
    # 先删除外围空格，并把末位小写 x 规范为大写。
    id_number = raw_id.strip().upper()
    # Validate the format before safely checking the checksum or extracting fields.
    # 必须先验证格式，之后才能安全地校验和提取。
    if not has_valid_format(id_number):
        # Display the English format error when that check fails.
        # 格式不通过时显示英文错误信息。
        print("Invalid format.")
        # Return early instead of entering the later checksum function.
        # 提前结束，避免进入后续校验函数。
        return
    # Compare the checksum only after the format has passed.
    # 仅在格式通过后比较校验位。
    if not has_valid_checksum(id_number):
        # Display the second kind of error when the check character does not match.
        # 校验位不符时显示第二类错误。
        print("Invalid checksum.")
        # Return early without printing the birth date or sex code.
        # 提前结束，不输出出生日期和性别码。
        return
    # Display success only after both checks have passed.
    # 只有两个检查都通过才显示成功信息。
    print("Format and checksum passed.")
    # Call the extraction function and display the birth date.
    # 调用提取函数并显示出生日期。
    print("Date of birth:", get_birth_date(id_number))
```

```
    # Call the parity-based function and display the sex code.
    # 调用奇偶判断函数并显示性别码。
    print("Sex:", get_sex(id_number))

# Preserve the source's complete example and keep its leading zeroes in a string.
# 保留原题给出的完整示例，字符串保存前导零。
raw_id = "00000020000101038X"
# Label the original PDF example and check it without changing it.
# 标明原 PDF 给出的例子，保留原样并如实检查。
print("Source example:")
# Under the stated formula, the original example should print Invalid checksum.
# 按原题公式运行，原例应显示 Invalid checksum.。
main(raw_id)
# Change only the final character to the calculated 2, and label this as a separate repair.
# 只把末位改为公式计算出的二，明确另列修正样例。
corrected_id = "000000200001010382"
# Clearly distinguish the corrected example from the inconsistent source example.
# 清楚区分修正样例与原题不一致的例子。
print("Corrected classroom-rule example:")
# The corrected example passes and displays success, the birth date, and Female.
# 修正样例通过后显示成功、出生日期和 Female。
main(corrected_id)

Source example:
Invalid checksum.
Corrected classroom-rule example:
Format and checksum passed.
Date of birth: 2000-01-01
Sex: Female
```

Worked answer

完整解答

Preserve leading zeroes by using a string. Validate the format before the checksum. Return the calculated check digit as a string.

使用字符串保留前导零。先验证格式，再验证校验和。计算得到的校验位要返回字符串。

The source sample does not pass its own stated rule: its weighted total is 46 and remainder is 2, so X is incorrect. The original sample prints Invalid checksum.

原样例不符合它自己给出的规则：加权总和为 46、余数为 2，因此 X 不正确。原样例打印 Invalid checksum.。

The explicitly corrected classroom-rule sample 000000200001010382 passes and yields 2000-01-01 and Female.

明确标出的课堂规则修正样例 000000200001010382 可以通过，并提取出 2000-01-01 和 Female。

Common mistakes

常见错误

Do not lose leading zeroes by converting the whole ID to an integer, extract fields before format validation, return an integer check digit for a string comparison, or silently replace the classroom formula.

不要把整串转整数而丢失前导零，不要在格式验证之前提取字段，不要返回整数校验位来与字符串比较，也不要悄悄替换课堂公式。

Check your understanding

检查理解

Question: Why does the year slice end at 10 rather than 9?

问题：为什么年份切片结束于 10，而不是 9？

Answer: Slices exclude the right endpoint. [6:10] contains indexes 6, 7, 8, and 9, giving four characters.

答案：切片不包括右端点。[6:10] 包含索引 6、7、8、9，共四个字符。

Closing review

结束前的回顾

The central skill is explaining what a function receives, what it returns, and when it stops. Our calculation exercises practiced parameters and return values, the string exercises practiced traversal and state, and the final programs combined helpers into a clear workflow.

核心能力是解释函数接收什么、返回什么、何时停止。计算题练习形参与返回值，字符串题练习遍历与状态，最后的程序把辅助函数组合成清晰流程。

Before we finish, answer three questions: Why is printing different from returning? Why must we consider zero and empty results? Why must validation come before extracting fixed fields?

结束前请回答三个问题：为什么打印不同于返回？为什么必须考虑零与空结果？为什么验证必须先于固定字段提取？

Printing displays a value, while returning provides it to the caller. Boundary cases can change whether a loop runs or a mean exists. Validation makes later indexing and calculations safe under the stated rules.

打印显示一个值，返回则把值提供给调用者。边界情况可能改变循环是否运行，或平均数是否存在。验证使后续索引与计算在约定规则下安全进行。